This chapter focuses on:

- The concept of a memory hierarchy
- Understanding cache memory
- Cache organization types: direct-mapped, set-associative, fully associative
- Locality of reference
- Hit/miss, hit rate, miss penalty
- Virtual memory
- Translation Lookaside Buffer (TLB)

📘 Chapter 7: Large and Fast – Exploiting Memory Hierarchy

🔹 Slide: Basic Structure of Memory Hierarchy

📌 Summary:

The memory hierarchy organizes different types of memory based on:

Property	Top Level	Bottom Level
Speed	Fastest	Slowest
Size	Smallest	Largest
Cost/bit	Highest	Lowest

From top to bottom:

1. Registers (CPU registers)
2. Cache (L1, L2, L3 caches)
3. Main Memory (RAM)
4. Secondary Storage (SSD, HDD)

🔹 **Slide: Locality – Why Memory Hierarchy Works**

📌 **Summary:**

There are two types of locality :

Type	Definition
Temporal Locality	If an item is accessed, it's likely to be accessed again soon
Spatial Locality	If an item is accessed, nearby items will soon be accessed

These principles justify having smaller, faster memory levels act as caches for larger, slower ones.

🧠 **Teaching Tip:**

Use code examples:

```
Loop:
  lw $t0, 0($s0)      # Load word
  addi $s0, $s0, 4
  bne $s0, $s1, Loop  # Repeat until end
```

→ Demonstrates spatial locality (accessing memory sequentially)

🔹 **Slide: Hit and Miss**

📌 **Summary:**

When accessing data in a cache:

- Hit: Data is found in the upper level
- Miss: Data must be fetched from the lower level

📈 **Performance Metrics:**

Metric	Description
Hit Rate	% of accesses that are hits
Miss Rate	1 – Hit Rate
Hit Time	Time to deliver data if hit
Miss Penalty	Time to fetch the block from the lower level + time to deliver

🎯 **Student Question:**

“Why is it important to minimize miss penalty?”

👉 Because memory access is much slower than CPU speed, and a miss causes the CPU to wait, wasting time.

🔹 **Slide: Caches and Block Transfers**

📌 **Summary:**

Data moves between memory levels in blocks (also called lines). When data is not in cache:

1. A block containing the requested data is fetched from main memory.
2. It replaces an existing block in the cache.
3. Data is then provided to the CPU.

🧠 **Example:**

Suppose you request a word at address X:

- The entire block (say, 64 bytes) around X is loaded into cache.

🎯 **Teaching Tip:**

Draw a simple diagram showing:

- A cache with multiple blocks
- A miss → fetch from main memory → replace the least recently used block

🔹 **Slide: Direct-Mapped Cache**

📌 **Summary:**

Each memory block maps to a unique location in the cache.

🔢 **Address Breakdown:**

In a direct-mapped cache, each memory address is divided into three parts:

Field	Purpose
Tag	Identifies which block from memory this is
Index	Specifies which cache line to look in
Offset	Specifies which byte within the block

Example:

- 32-bit address
- 10-bit index → 1024 cache lines
- Remaining bits split into tag (20 bits) and offset (2 bits for 4-byte words)

🔄 **How It Works:**

- Compare the requested tag with the stored tag at the index
- If match → cache hit

- Else → cache miss

⚠️ **Limitation:**

- Can lead to conflict misses if two frequently used blocks map to the same index

🔹 **Slide: Fully Associative & Set-Associative Caching**

📌 **Summary:**

To reduce conflict misses, we increase associativity :

Type	Description	Pros	Cons
Direct Mapped	One unique location per block	Simple, fast	More conflict misses
Fully Associative	Any block can go anywhere	Minimizes conflict misses	Complex, slow
Set-Associative	Each block fits in one locations	Balance between speed and performance	More complex than a direct mapping

🧠 **Teaching Tip:**

Compare to:

- Locker room lockers (direct mapped): only one locker per person
- Gym bag storage system (set-associative): a few specific spots per user
- Free-for-all shelves (fully associative): any package can go anywhere

🔹 **Slide: Virtual Memory Introduction**

📌 **Summary:**

Virtual memory gives each program its own virtual address space, independent of physical memory layout.

Key Concepts:

- Page: Fixed-size block of memory (e.g., 4KB)
- Page Table: Maps virtual pages to physical frames
- Page Fault: Occurs when the page is not in RAM (must load from disk)

📦 **Analogy:**

Think of virtual memory like a bookshelf that spills over to basement storage :

- You only keep what you're using on the shelf (RAM).
- When needed, you retrieve books from the basement (disk).

🔹 **Slide: Page Tables and TLB**

📌 **Summary:**

- Every memory access goes through the page table to translate a virtual address to a physical address.
- Page tables reside in memory and can be very large.
- To speed up translation, CPUs use a small, fast cache called the Translation Lookaside Buffer (TLB).

🧩 **Address Translation:**

A virtual address is broken into:

- Page Number (used to index TLB/page table)
- Page Offset (offset within the page)

🎯 **Teaching Points:**

- TLB hit: address translation is fast
- TLB miss: need to access page table in memory → slow
- TLB helps reduce average memory access time

🔹 **Slide: Cache Write Policies**

📌 **Summary:**

Two main strategies for handling writes:

1. Write-through: Always write to both cache and main memory
2. Write-back: Only write to cache; defer memory update until cache block is replaced

📝 **Variations:**

- Write allocate: On write miss, bring the block into the cache
- No-write allocate: On write miss, directly write to memory

🧠 **Design Trade-off:**

Policy	Pros	Cons
Write-through	Simpler, consistent memory	Slower due to frequent writes
Write-back	Faster, less memory traffic	Risk of inconsistency, more complex

🎯 **Suggested Question:**

“What happens if a system crashes right after a write-back but before writing to memory?”

🔹 **Slide: Reducing Miss Rates with Associative Mapping**

📌 **Summary:**

As associativity increases:

- Miss rate decreases
- Hardware complexity increases

⚖️ **Trade-offs:**

Associativity	Miss Rate	Complexity	Lookup Time
Direct-mapped	High	Low	Fast
Set-associative	Medium	Medium	Moderate
Fully associative	Low	High	Slow

📊 **Example:**

With 8 cache blocks:

- Direct-mapped: block 12 → always index 4
- 2-way set-associative: block 12 → index 0
- Fully associative: block 12 → any available slot

🔹 **Slide: Virtual Memory and Page Fault Handling**

📌 **Summary:**

- Programs work with virtual addresses
- The MMU (Memory Management Unit) translates them to physical addresses
- If the page isn't in RAM, → Page Fault → OS loads it from disk

🧠 **Diagram Tip:**

Show how a page fault leads to:

1. OS suspends the current process
2. Fetches page from disk
3. Update pthe age table
4. Resumes execution

🎯 **Suggested Question:**

"Why is handling a page fault done in software rather than hardware?"

➢ Because disk access is so slow that hardware would be too costly to implement all the logic needed.

🔹 **Slide: TLB – Improving Virtual Memory Performance**

📌 **Summary:**

Every virtual → physical address translation requires reading from the page table in memory.

To avoid slowing down every memory access:

- We use a TLB (Translation Lookaside Buffer), which is a cache of page table entries.

🧠 **How It Works:**

1. The processor uses a virtual address
2. TLB checked first
    - TLB hit → quick translation
    - TLB miss → walk page table in memory

🎯 **Suggested Question:**

“How does the TLB help reduce memory access time?”

🔹 **Slide: Trends in Memory Technology**

📌 **Summary:**

Processor speeds are increasing faster than memory/disk speeds → growing performance gap.

Solutions include:

- Using burst mode SRAMs
- Optimizing DRAM for higher bandwidth
- Prefetching data before it's needed
- Better compiler-level data reuse and locality optimization

🧠 **Real-world Connection:**

Modern systems combine:

- Multiple cache levels (L1, L2, L3)
- Smart caching algorithms (LRU, pseudo-LRU)
- Hardware prefetching

✅ **Summary Table**

Concept	Explanation
Memory Hierarchy	Organizes memory from fastest/smallest to slowest/largest
Locality	Temporal and spatial patterns justify caching
Cache Types	Direct-mapped, set-associative, fully associative
TLB	Fast cache for virtual-to-physical address translation
Virtual Memory	Gives the illusion of a large memory using the disk as a backing store

Page Fault	When data is not in memory, it needs to be loaded from disk
Cache Miss Penalties	Significantly impacts performance if not managed well